

Exercice Pratique : Application Multi-Conteneurs**Contexte :**

Tu vas dockeriser une petite application web qui utilise un serveur Nginx pour le frontend et une base de données PostgreSQL pour le backend. Tu devras utiliser un Dockerfile pour créer une image personnalisée pour le serveur, un fichier `.dockerignore` pour optimiser le processus de construction, et un fichier `docker-compose.yml` pour orchestrer les services.

Étape 1 : Préparer l'application

1. Crée une structure de répertoires comme suit :

`project/`

```
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── .dockerignore
├── backend/
│   └── db.env
└── docker-compose.yml
```

2. Dans le dossier `frontend/`, crée un fichier `index.html` avec le contenu suivant :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dockerized App</title>
</head>
<body>
  <h1>Bienvenue dans l'application Dockerisée</h1>
</body>
</html>
```

Étape 2 : Créer un Dockerfile

1. Dans `frontend/Dockerfile`, écris :

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

2. Dans `frontend/.dockerignore`, ajoute :

```
*.log
```

Étape 3 : Configurer la base de données

1. Dans `backend/db.env`, configure les variables d'environnement :

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp
```

Étape 4 : Créer un fichier docker-compose.yml

1. Dans `docker-compose.yml`, définis les services comme suit :

```
version: "3.8"
services:
  frontend:
    build:
```

```
context: ./frontend
ports:
  - "8080:80"
networks:
  - app-network

db:
  image: postgres:13
  environment:
    - POSTGRES_USER=${POSTGRES_USER}
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - POSTGRES_DB=${POSTGRES_DB}
  env_file:
    - ./backend/db.env
  volumes:
    - db-data:/var/lib/postgresql/data
  networks:
    - app-network

pgadmin:
  image: dpage/pgadmin4
  environment:
    - PGADMIN_DEFAULT_EMAIL=admin@admin.com
    - PGADMIN_DEFAULT_PASSWORD=admin
  ports:
    - "5050:80"
  depends_on:
    - db
  networks:
    - app-network

volumes:
  db-data:
networks:
  app-network:
```

Étape 5 : Tester et valider

1. Construis les conteneurs :
`docker-compose up --build`
2. Vérifie que le frontend est accessible sur `http://localhost:8080`.
3. Connecte-toi à pgAdmin via `http://localhost:5050` avec les identifiants :
 - Email : `admin@admin.com`
 - Mot de passe : `admin`
4. Ajoute une connexion dans pgAdmin en utilisant les informations suivantes :
 - Host: `db`
 - Port: `5432`
 - Username: `admin`
 - Password: `adminpassword`
5. Ajoute un volume persistant pour la base de données et redémarre le conteneur.